从汇编层看64位程序运行——函数的调用和栈平衡

原创 breaksoftware 已于 2024-07-19 10:45:39 修改 阅读量1.5k 收藏 9 点赞数 41 CC 4.0 BY-SA版权

分类专栏： 从汇编层看64位程序运行 文章标签： 汇编

函数调用

不知道有没有人想过一个问题：A函数调用B函数，B函数是如何知道在调用结束后回到A函数中的？

比如下面的代码，main函数调用foo。当foo执行完毕，需要执行main函数的return 0语句。但是main和foo是割裂的，foo是怎么回到main函数中继续执行的呢？

c AI写代码 | 复制 运行

```c
void foo() {
    int a = 10;
    a = a + 5;
}

int main() {
    foo();
    return 0;
}
```

我们对上述代码进行反汇编。
在main函数中，我们没有看到任何蛛丝马迹，只看到call指令。

```
(gdb) disassemble
Dump of assembler code for function main:
   0x0000555555555352 <+0>:     endbr64
   0x0000555555555356 <+4>:     push   %rbp
   0x0000555555555357 <+5>:     mov    %rsp,%rbp
=> 0x000055555555535a <+8>:     mov    $0x0,%eax
   0x000055555555535f <+13>:    call   0x555555555129 <foo>
   0x0000555555555364 <+18>:    mov    $0x0,%eax
   0x0000555555555369 <+23>:    pop    %rbp
   0x000055555555536a <+24>:    ret
End of assembler dump.
```

CSDN @breaksoftware

上图显示，当前代码执行到+8处（但是此行并没有执行，它的上一行执行了）。我们需要注意它的地址以及其后的地址0x0000555555555364，即+18处。理论上说，foo函数执行完毕后，CPU会执行+18处的代码。我们将通过调试来查看具体的原因。

让程序运行到+13处，可以看到当前rsp和rbp值相同，都是0x7fffffffdf10。

```
(gdb) disassemble
Dump of assembler code for function main:
   0x0000555555555352 <+0>:     endbr64
   0x0000555555555356 <+4>:     push   %rbp
   0x0000555555555357 <+5>:     mov    %rsp,%rbp
   0x000055555555535a <+8>:     mov    $0x0,%eax
=> 0x000055555555535f <+13>:    call   0x555555555129 <foo>
   0x0000555555555364 <+18>:    mov    $0x0,%eax
   0x0000555555555369 <+23>:    pop    %rbp
   0x000055555555536a <+24>:    ret
End of assembler dump.
(gdb) p $rip
$1 = (void (*)()) 0x55555555535f <main+13>
(gdb) p $rsp
$2 = (void *) 0x7fffffffdf10
(gdb) p $rbp
$3 = (void *) 0x7fffffffdf10
(gdb)
```

CSDN @breaksoftware

进入foo函数内部，可以看到call指令导致rsp减少了4字节，即有一个4字节值被call指令push到栈中。

```
(gdb) si
0x0000555555555129 in foo ()
(gdb) disassemble
Dump of assembler code for function foo:
=> 0x0000555555555129 <+0>:     endbr64
   0x000055555555512d <+4>:     push   %rbp
   0x000055555555512e <+5>:     mov    %rsp,%rbp
   0x0000555555555131 <+8>:     movl   $0xa,-0x4(%rbp)
   0x0000555555555138 <+15>:    addl   $0x5,-0x4(%rbp)
   0x000055555555513c <+19>:    nop
   0x000055555555513d <+20>:    pop    %rbp
   0x000055555555513e <+21>:    ret
End of assembler dump.
(gdb) p $rip
$4 = (void (*)()) 0x555555555129 <foo>
(gdb) p $rsp
$5 = (void *) 0x7fffffffdf08
(gdb) p $rbp
$6 = (void *) 0x7fffffffdf10
```

CSDN @breaksoftware

通过下面指令，我们看到被push进来的值是0x0000555555555364。

```
(gdb) x /2xg $rsp
0x7fffffffdf08: 0x0000555555555364      0x0000000000000001
```

CSDN @breaksoftware

我们反汇编这个地址附近代码，可以发现call指令将当前函数返回到的代码地址给push到栈中。这样foo函数在执行完毕后，就可以在栈中取出该地址，从而跳转

到此处继续执行，完成从foo函数到main函数的跳跃。

```
(gdb) disassemble 0x0000555555555364
Dump of assembler code for function main:
   0x0000555555555352 <+0>:     endbr64
   0x0000555555555356 <+4>:     push   %rbp
   0x0000555555555357 <+5>:     mov    %rsp,%rbp
   0x000055555555535a <+8>:     mov    $0x0,%eax
   0x000055555555535f <+13>:    call   0x555555555129 <foo>
   0x0000555555555364 <+18>:    mov    $0x0,%eax
   0x0000555555555369 <+23>:    pop    %rbp
   0x000055555555536a <+24>:    ret
End of assembler dump.
```

CSDN @breaksoftware

栈平衡

栈平衡是指函数调用子函数后，栈顶要回到没有准备调用子函数之前。
在函数调用前后，涉及压栈操作的有如下情况：

- call指令会将本函数中下条要执行的指令地址push到栈中
- 如果子函数超过6个参数，则从第7个参数开始的参数会通过栈来传递

以下面的代码为例，

c　　AI写代码 | 复制　▷ 运行

```c
void foo10(int a, int b, int c, int d, int e, int f, int g, int h, int i, int j) {
    a = a + 5;
    b = b + 5;
    c = c + 5;
    d = d + 5;
    e = e + 5;
    f = f + 5;
    g = g + 5;
    h = h + 5;
```

```
    i = i + 5;
    j = j + 5;
    int sum = a + b + c + d + e + f + g + h + i + j;
    sum = sum + 5;
}

int main() {
    int a = 10;
    int b = 20;
    int c = 30;
    int d = 40;
    int e = 50;
    int f = 60;
    int g = 70;
    int h = 80;
    int i = 90;
    int j = 100;
    foo10(a, b, c, d, e, f, g, h, i, j);
    return 0;
}
```

收起 ^

其中main函数的反汇编结果如下

```
(gdb) disassemble
Dump of assembler code for function main:
   0x0000555555555352 <+0>:     endbr64
   0x0000555555555356 <+4>:     push   %rbp
   0x0000555555555357 <+5>:     mov    %rsp,%rbp
=> 0x000055555555535a <+8>:     sub    $0x30,%rsp
   0x000055555555535e <+12>:    movl   $0xa,-0x28(%rbp)
   0x0000555555555365 <+19>:    movl   $0x14,-0x24(%rbp)
   0x000055555555536c <+26>:    movl   $0x1e,-0x20(%rbp)
   0x0000555555555373 <+33>:    movl   $0x28,-0x1c(%rbp)
   0x000055555555537a <+40>:    movl   $0x32,-0x18(%rbp)
   0x0000555555555381 <+47>:    movl   $0x3c,-0x14(%rbp)
   0x0000555555555388 <+54>:    movl   $0x46,-0x10(%rbp)
   0x000055555555538f <+61>:    movl   $0x50,-0xc(%rbp)
   0x0000555555555396 <+68>:    movl   $0x5a,-0x8(%rbp)
   0x000055555555539d <+75>:    movl   $0x64,-0x4(%rbp)
   0x00005555555553a4 <+82>:    mov    -0x14(%rbp),%r9d
   0x00005555555553a8 <+86>:    mov    -0x18(%rbp),%r8d
   0x00005555555553ac <+90>:    mov    -0x1c(%rbp),%ecx
   0x00005555555553af <+93>:    mov    -0x20(%rbp),%edx
   0x00005555555553b2 <+96>:    mov    -0x24(%rbp),%esi
   0x00005555555553b5 <+99>:    mov    -0x28(%rbp),%eax
   0x00005555555553b8 <+102>:   mov    -0x4(%rbp),%edi
   0x00005555555553bb <+105>:   push   %rdi
   0x00005555555553bc <+106>:   mov    -0x8(%rbp),%edi
   0x00005555555553bf <+109>:   push   %rdi
   0x00005555555553c0 <+110>:   mov    -0xc(%rbp),%edi
   0x00005555555553c3 <+113>:   push   %rdi
   0x00005555555553c4 <+114>:   mov    -0x10(%rbp),%edi
   0x00005555555553c7 <+117>:   push   %rdi
   0x00005555555553c8 <+118>:   mov    %eax,%edi
   0x00005555555553ca <+120>:   call   0x5555555552d4 <foo10>
```

```
   0x00005555555553cf <+125>:   add    $0x20,%rsp
   0x00005555555553d3 <+129>:   mov    $0x0,%eax
   0x00005555555553d8 <+134>:   leave
   0x00005555555553d9 <+135>:   ret
End of assembler dump.
```

CSDN @breaksoftware

我们来通过动态调试，来查看上述代码如何做到栈平衡。

我们让代码执行到+105处，查看此时rsp寄存器和栈上数据

```
(gdb) b *0x00005555555553bb
Breakpoint 2 at 0x5555555553bb
(gdb) c
Continuing.

Breakpoint 2, 0x00005555555553bb in main ()
(gdb) p $rsp
$1 = (void *) 0x7fffffffdef0
(gdb) x /4xg $rsp
0x7fffffffdef0: 0x0000000000000002      0x000000140000000a
0x7fffffffdf00: 0x000000280000001e      0x0000003c00000032
```

CSDN @breaksoftware

然后让其他参数入栈，可以看到栈增加了0x20。（即rsp的值减少了0x20，0x7fffffffdef0->0x7fffffffded0）

```
(gdb) b *0x00005555555553c8
Breakpoint 3 at 0x5555555553c8
(gdb) c
Continuing.

Breakpoint 3, 0x00005555555553c8 in main ()
(gdb) p $rsp
$2 = (void *) 0x7fffffffded0
(gdb) x /8xg $rsp
0x7fffffffded0: 0x0000000000000046      0x0000000000000050
0x7fffffffdee0: 0x000000000000005a      0x0000000000000064
0x7fffffffdef0: 0x0000000000000002      0x000000140000000a
0x7fffffffdf00: 0x000000280000001e      0x0000003c00000032
(gdb)
```

CSDN @breaksoftware

进入foo10函数后，call导致的Next RIP压栈会在其ret时弹出。于是foo10执行完毕回到main函数后，rsp的值还不处于栈平衡状态。

```
(gdb) b *0x00005555555553cf
Breakpoint 4 at 0x5555555553cf
(gdb) c
Continuing.

Breakpoint 4, 0x00005555555553cf in main ()
(gdb) p $rsp
$3 = (void *) 0x7fffffffded0
```

CSDN @breaksoftware

但是紧接着foo10函数的调用，main函数会通过下面的命令让栈获得平衡

```
   0x00005555555553ca <+120>:   call   0x5555555552d4 <foo10>
=> 0x00005555555553cf <+125>:   add    $0x20,%rsp
   0x00005555555553d3 <+129>:   mov    $0x0,%eax
   0x00005555555553d8 <+134>:   leave
   0x00005555555553d9 <+135>:   ret
```

CSDN @breaksoftware

```
(gdb) si
0x00005555555553d3 in main ()
(gdb) p $rsp
$4 = (void *) 0x7fffffffdef0
```

CSDN @breaksoftware

总体来说，**栈的失衡是函数调用导致的，且发生在调用者函数中；栈的再平衡则是在子函数退出，执行回到调用者函数时立即执行的。**